

Manual para subir y mantener la aplicación ELECIND

Jhorman, te dejo este manual para que puedas trabajar con la aplicación **ELECIND / uceda-bellon** en el servidor sin depender de nadie para cada cambio. La idea es que puedas subir actualizaciones completas cuando tengas una nueva versión del proyecto, pero también que puedas entrar y modificar archivos concretos si lo necesitas como desarrollador.

La aplicación ya está instalada y funcionando en producción. La URL pública es:

Plain Text

```
https://elecind.getradi.es/login
```

1. Datos principales del servidor

Estos son los datos que vas a necesitar para conectarte y trabajar con el proyecto.

Dato	Valor
Servidor	212.227.169.102
Usuario SSH	jhorman
Carpeta de subida de ZIP	/home/jhorman/uploads
Carpeta real del proyecto	/opt/entredes/apps/uceda-bellon/current
URL pública	https://elecind.getradi.es/login
Comando de despliegue	sudo /usr/local/sbin/deploy-uceda-bellon /home/jhorman/uploads/archivo.zip

Tu usuario tiene permisos para trabajar sobre los archivos del proyecto. También tiene permisos limitados para ejecutar el despliegue y consultar algunos servicios necesarios, pero no tiene permisos de administración completa del servidor.

2. Cómo conectarte por SSH

Para entrar al servidor, abre una terminal en tu ordenador. Si estás en Windows puedes usar **Windows Terminal**, **PowerShell**, **Git Bash** o la terminal integrada de VS Code.

Ejecuta:

Bash

```
ssh jhorman@212.227.169.102
```

Si es la primera vez que conectas, puede aparecer una pregunta parecida a esta:

Plain Text

Are you sure you want to continue connecting?

Responde:

Plain Text

yes

Después entrarás al servidor con el usuario `jhorman`.

3. Cómo ir a la carpeta del proyecto

Una vez dentro del servidor, entra en la carpeta real de la aplicación:

Bash

```
cd /opt/entredes/apps/uceda-bellon/current
```

Para comprobar que estás en la carpeta correcta, ejecuta:

Bash

```
ls -la
```

Deberías ver archivos y carpetas típicos de Laravel, como:

Plain Text

```
app
bootstrap
config
database
public
resources
routes
storage
artisan
```

```
composer.json
package.json
vite.config.js
```

Si ves eso, estás en el sitio correcto.

4. Qué puedes tocar y qué debes tratar con cuidado

Puedes modificar los archivos del proyecto porque eres el desarrollador responsable. Aun así, hay archivos y carpetas que conviene tratar con cuidado porque afectan directamente a producción.

Elemento	Puedes modificarlo	Comentario
app/	Sí	Código PHP de la aplicación.
resources/	Sí	Vistas, CSS, JS y Blade.
routes/	Sí	Rutas web, móvil y API.
database/migrations/	Sí, con cuidado	Si subes migraciones, el despliegue las ejecutará.
config/	Sí, con cuidado	Puede afectar al comportamiento global.
public/	Sí	Archivos públicos, imágenes, assets, etc.
.env	Sí, pero con mucho cuidado	Contiene configuración sensible de producción. No lo borres ni lo sobrescribas desde local.
storage/	Solo si sabes lo que haces	Contiene logs, archivos y datos generados por la aplicación.
vendor/	No manualmente	Lo gestiona Composer.
node_modules/	No manualmente	Lo gestiona npm.

La norma práctica es sencilla: si vas a cambiar código, vistas, rutas, controladores, modelos o migraciones, puedes hacerlo. Si vas a tocar `.env` , `storage` , permisos, base de datos, servidor web o servicios, avísame antes o asegúrate de tener claro lo que estás haciendo.

5. Forma recomendada de subir una versión completa

Cuando tengas una nueva versión lista, lo más limpio es subir un ZIP del proyecto y ejecutar el script de despliegue. Ese script hace todo lo necesario para que Laravel quede bien actualizado.

El ZIP debe contener la carpeta raíz del proyecto Laravel. Dentro deben estar archivos como:

Plain Text

```
artisan
composer.json
composer.lock
package.json
package-lock.json
app/
bootstrap/
config/
database/
public/
resources/
routes/
```

No hace falta incluir `vendor` ni `node_modules`, porque el servidor los genera automáticamente.

Tampoco debes incluir el `.env` local para no sobrescribir el `.env` de producción.

6. Cómo subir el ZIP al servidor

Desde tu ordenador, estando en la carpeta donde tengas el ZIP, ejecuta:

Bash

```
scp uceda-bellon-main.zip jhorman@212.227.169.102:/home/jhorman/uploads/
```

Si tu archivo tiene otro nombre, cambia `ucedas-bellon-main.zip` por el nombre real. Por ejemplo:

Bash

```
scp elecind-nueva-version.zip jhorman@212.227.169.102:/home/jhorman/uploads/
```

Después entra al servidor:

Bash

```
ssh jhorman@212.227.169.102
```

Comprueba que el archivo está subido:

```
Bash
```

```
ls -lh /home/jhorman/uploads
```

7. Cómo ejecutar el despliegue

Una vez subido el ZIP, ejecuta el despliegue con este comando:

```
Bash
```

```
sudo /usr/local/sbin/deploy-uceda-bellon /home/jhorman/uploads/uceda-bellon-mai
```

Si tu ZIP tiene otro nombre, cambia la ruta. Por ejemplo:

```
Bash
```

```
sudo /usr/local/sbin/deploy-uceda-bellon /home/jhorman/uploads/elecind-nueva-ve
```

El despliegue irá mostrando pasos como estos:

```
Plain Text
```

```
[1/10] Backup previo
[2/10] Extrayendo ZIP
[3/10] Sincronizando código sin sobrescribir .env ni storage
[4/10] Composer install
[5/10] Frontend build
[6/10] Migraciones y enlaces
[7/10] Cachés Laravel
[8/10] Permisos para desarrollador y proceso web
[9/10] Reinicio servicios app
[10/10] Limpieza temporal
OK despliegue completado
```

Si termina con `OK despliegue completado`, la subida ha terminado correctamente.

8. Qué hace el script de despliegue

El script no se limita a copiar archivos. Hace una actualización completa de Laravel para evitar errores típicos de producción.

Paso	Qué hace
Backup previo	Guarda una copia de la versión anterior antes de tocar nada.
Extracción del ZIP	Descomprime la nueva versión en una carpeta temporal.
Sincronización	Copia el código nuevo sin sobrescribir <code>.env</code> , <code>storage</code> , <code>vendor</code> , <code>node_modules</code> ni <code>public/storage</code> .
Composer	Ejecuta dependencias PHP de producción.
npm	Instala dependencias frontend.
Build	Ejecuta <code>npm run build</code> para generar assets.
Migraciones	Ejecuta <code>php artisan migrate --force</code> .
Cachés	Regenera cachés de configuración, rutas y vistas.
Permisos	Deja el proyecto editable por tu usuario y utilizable por el servidor web.
Reinicio	Reinicia PHP-FPM, Nginx y la cola de Laravel.

Esto evita tener que ejecutar muchos comandos manuales cada vez.

9. Cómo modificar un archivo directamente

Si necesitas tocar un archivo concreto, puedes hacerlo directamente por SSH, SFTP, WinSCP, FileZilla o VS Code Remote SSH.

Por SSH, primero entra:

Bash

```
ssh jhorman@212.227.169.102
```

Después ve al proyecto:

Bash

```
cd /opt/entredes/apps/uceda-bellon/current
```

Puedes editar un archivo con `nano` . Por ejemplo:

Bash

```
nano resources/views/web/dashboard.blade.php
```

Para guardar en `nano` :

Plain Text

Ctrl + O

Enter

Ctrl + X

Si cambias archivos PHP, Blade o configuración, puede que necesites limpiar cachés. Puedes hacerlo desde la carpeta del proyecto:

Bash

```
php artisan optimize:clear  
php artisan config:cache  
php artisan route:cache  
php artisan view:cache
```

Si cambias JavaScript, CSS, Tailwind o Vite, normalmente debes recompilar assets:

Bash

```
npm run build
```

Después comprueba la web en:

Plain Text

```
https://elecind.getradi.es/login
```

10. Comandos útiles del proyecto

Dentro de la carpeta del proyecto:

Bash

```
cd /opt/entrededados/apps/uceda-bellon/current
```

Ver estado de migraciones:

Bash

```
php artisan migrate:status
```

Ejecutar migraciones pendientes:

Bash

```
php artisan migrate --force
```

Limpiar cachés:

Bash

```
php artisan optimize:clear
```

Regenerar cachés de producción:

Bash

```
php artisan config:cache  
php artisan route:cache  
php artisan view:cache
```

Compilar frontend:

Bash

```
npm run build
```

Ver logs recientes de Laravel:

Bash

```
tail -n 100 storage/logs/laravel.log
```

Ver logs en directo:

Bash

```
tail -f storage/logs/laravel.log
```

Para salir del log en directo:

Plain Text

Ctrl + C

11. Comprobar servicios principales

Puedes comprobar si la cola de Laravel está activa:

Bash

```
sudo systemctl status uceda-bellon-queue.service
```

Puedes comprobar Nginx:

Bash

```
sudo systemctl status nginx
```

Puedes comprobar PHP-FPM:

Bash

```
sudo systemctl status php8.3-fpm
```

Cuando estés viendo una pantalla de `systemctl status`, para salir pulsa:

Plain Text

q

12. Qué hacer si la web da error 500

Si aparece un error 500, lo primero es mirar el log de Laravel:

Bash

```
cd /opt/entredes/apps/uceda-bellon/current
tail -n 100 storage/logs/laravel.log
```

Si el error es por caché, prueba:

Bash

```
php artisan optimize:clear
php artisan config:cache
php artisan route:cache
php artisan view:cache
```

Si el error aparece justo después de subir una versión, revisa especialmente:

Posible causa	Qué revisar
Error PHP	storage/logs/laravel.log
Error de migración	Última migración añadida en database/migrations
Error frontend	Resultado de npm run build
Error de dependencia PHP	Resultado de composer install
Error de permisos	Si Laravel puede escribir en storage y bootstrap/cache

No repitas despliegues muchas veces sin revisar el error. Si falla, copia el error exacto y me lo pasas.

13. Qué hacer si falla npm run build

Si el despliegue falla en npm run build, normalmente el problema está en JavaScript, CSS, Tailwind, Vite o imports del frontend.

Antes de subir una versión, si has tocado frontend, conviene probar en local:

Bash

```
npm install
npm run build
```

Si en local falla, también fallará en el servidor.

14. Qué hacer si falla composer install

Si falla `composer install`, suele haber un problema con `composer.json`, `composer.lock` o alguna extensión PHP necesaria.

Antes de subir una versión que cambie dependencias PHP, prueba en local:

Bash

```
composer install
```

Si has añadido paquetes, asegúrate de subir también `composer.lock` actualizado.

15. Qué hacer si hay migraciones nuevas

Si añades migraciones en `database/migrations`, el script de despliegue las ejecutará automáticamente con:

Bash

```
php artisan migrate --force
```

Antes de subir una migración, pruébala en local o en entorno de pruebas. Una migración mal hecha puede modificar datos de producción.

Si tienes dudas con una migración destructiva, avísame antes de subirla.

16. Backups antes de cada despliegue

El script crea un backup antes de sincronizar la nueva versión. Los backups se guardan en:

Plain Text

```
/opt/entredes/apps/uceda-bellon/backups
```

Puedes verlos con:

Bash

```
ls -lh /opt/entredes/apps/uceda-bellon/backups
```

No borres backups sin confirmarlo antes.

17. Flujo recomendado para cada subida

Este es el flujo que quiero que sigas en una subida normal.

Primero, prepara el ZIP en tu ordenador. Después súbelo:

Bash

```
scp uceda-bellon-main.zip jhorman@212.227.169.102:/home/jhorman/uploads/
```

Entra al servidor:

Bash

```
ssh jhorman@212.227.169.102
```

Ejecuta el despliegue:

Bash

```
sudo /usr/local/sbin/deploy-uced-bellon /home/jhorman/uploads/uced-bellon-mai
```

Comprueba la cola:

Bash

```
sudo systemctl status uceda-bellon-queue.service
```

Comprueba la web:

Plain Text

```
https://elecind.getradi.es/login
```

Si todo está bien, avísame con un mensaje corto indicando que el despliegue está hecho.

18. Mensaje que quiero que me envíes antes y después de desplegar

Antes de desplegar, avísame así:

Plain Text

```
Voy a desplegar nueva versión de ELECIND.  
Archivo: [nombre del ZIP]
```

```
Cambios principales: [resumen]
Incluye migraciones: sí/no
Incluye cambios frontend: sí/no
```

Después de desplegar:

Plain Text

```
Despliegue completado.
URL comprobada: https://elecind.getradi.es/login
Resultado: OK / incidencia
Observaciones: [lo que corresponda]
```

19. Cosas que no quiero que hagas sin avisarme

Puedes trabajar sobre el proyecto, pero hay ciertas acciones que prefiero que no hagas sin avisar antes porque pueden afectar al servidor completo o a datos de producción.

Acción	Avisar antes
Borrar base de datos	Sí
Ejecutar migraciones destructivas	Sí
Tocar configuración de Nginx	Sí
Tocar configuración de Traefik	Sí
Borrar <code>storage</code>	Sí
Borrar backups	Sí
Cambiar <code>.env</code> de producción	Sí
Instalar paquetes del sistema con <code>apt</code>	Sí
Reiniciar el servidor completo	Sí

La idea es que tengas libertad para desarrollar y subir cambios, pero sin tocar partes del servidor que afectan a otras aplicaciones o servicios.

20. Resumen rápido

Para entrar:

Bash

```
ssh jhorman@212.227.169.102
```

Para ir al proyecto:

```
Bash
```

```
cd /opt/entredes/apps/uceda-bellon/current
```

Para subir ZIP desde tu ordenador:

```
Bash
```

```
scp uceda-bellon-main.zip jhorman@212.227.169.102:/home/jhorman/uploads/
```

Para desplegar:

```
Bash
```

```
sudo /usr/local/sbin/deploy-uceda-bellon /home/jhorman/uploads/uceda-bellon-main.zip
```

Para ver logs:

```
Bash
```

```
tail -n 100 /opt/entredes/apps/uceda-bellon/current/storage/logs/laravel.log
```

Para comprobar la cola:

```
Bash
```

```
sudo systemctl status uceda-bellon-queue.service
```

Para comprobar la web:

```
Plain Text
```

```
https://elecind.getradi.es/login
```